

Faiza

26K-0019

Qno: 1

Take a number and print numbers from N down to 1.

Algorithm:

1. Start

2. Read an integer input N .

3. Set a loop counter $current = N$.

4. While $current$ is greatest than or equal to 1 -

a. display $current$ -

b. Decrement $current$ by 1
($current = current - 1$)

5. End.

Pseudocode:

Begin

Read in

for $current$ from N down to 1

Print $current$

end for

End -

Qno: 2

Input a year and determine whether it is a leap year.

Rule: divisible by 4 and (not divisible by 100 or divisible by 400).

Algorithm:

1. Start

2. Put an integer input year

3. Check the condition

• If $(Year \% 4 == 0) \text{ AND } (Year \% 100 \neq 0 \text{ OR } Year \% 400 == 0)$

• Display leap year

• Else

• Display "Not leap year"

4. End -

Pseudocode:

Begin

Input Year

If $(Year \text{ MOD } 4 == 0) \text{ AND } (Year \text{ MOD } 100 \neq 0 \text{ OR } Year \text{ MOD } 400 == 0)$ then

Print "leap year"

Else

Print "Not a leap year"

End If

End -

Question 3

Take a number, count how many digits it has, and state whether it is a single-digit, double-digit or triple-digit or more number.

Algorithm:

1. Start
2. Read an integer input, num
3. Store value of num in a variable temp.
4. If $\text{temp} == 0$, set $\text{count} = 1$
5. Otherwise
 - a. Initialize $\text{count} = 0$
 - b. While $\text{temp} > 0$
 - Increment count by 1
 - Perform integer division
 $\text{temp} = \text{temp} / 10$
6. Display digit count (count)
7. Determine classification based on count
 - If $\text{count} == 1$, display "single digit number"
 - Else if $\text{count} == 2$, display "Double Digit number"

Else count ≥ 3), display "Triple digit or more number"
B. End

Pseudocode
Begin

Read num
temp = (num)
If temp $\neq 0$ Then
 count = 1

Else

 count = 0

 While temp > 0 Do

 count = count + 1

 temp = temp / 10

 End while

End of

Print "Digit count: ", count

If count = 1 Then

 Print "Single digit number"

Else if count = 2 Then Print
 "Double digit number"

else Print "Three digit number or more number"

End-

~~Algo:~~

Question 4.

Print the multiplication tables from 1 to 10, but skip the table of any number that is a multiple of 3.

Algorithm:

1. start

2. loop variable i from 1 to 10.

a. if $i \% 3 == 0$ (i is multiple of 3)

• skip to next number of i loop

b. loop variable j from 1 to 10.

• Calculate product $= i \times j$

• Display $i \times j = \text{product}$

c. Print a new line for visual separation

3. End-

Pseudocode:

Begin

for i from 1 to 10 Do

If $i \text{ MOD } 3 = 0$ Then

Continue // skip multiplication
table for multiples of 3

End if

Print

for j from 1 to 10 Do

Product = $i * j$

Print $i + " * " + j + " = " + \text{product}$

End for

Print

End for

End

Question 5

Take a number, repeatedly sum its digits until the result is a single-digit.

Algorithm:

1. Start
2. Read an integer input num -
3. Take the absolute value of num
(Current = num)
4. Display initial value of current -

5. While $\text{current} \geq 10$:

a. Initialize $\text{sum} = 0$

b. Set $\text{temp} = \text{current}$

c. While $\text{temp} > 0$

- Extract last digit: $\text{digit} = \text{temp} \% 10$

- Add to sum: $\text{sum} = \text{sum} + \text{digit}$

- Reduce temp: $\text{temp} = \text{temp} / 10$

- Update $\text{current} = \text{sum}$

- Display intermediate result
current -

6. Display final single - digit result

7. End -

Pseudocode:

Begin

Read num

current = num

Print "initial number: ", current

while current ≥ 10 Do

sum = 0

temp = current

While temp > 0 Do

digit = temp MOD 10

sum = sum + digit

temp = temp DIV 10

End While

current = sum

Print "intermediate sum: ", current

End While

Print "final single-digit result: ", current

End -

PART - B

Question 1: (Hotel Booking) system

1. PAC:

Data input	Processing	Output	Given
Season: Peak [1] or Off Peak [2]	Determine base rate per night based on season and room type	Base rate per night	Peak season rate: standard: 5000 Deluxe: 8000 Suite: 12,000
Room type: Standard [1] Deluxe [2] Suite [3]	Calculate base cost = rate x nights	Discount amount	Off Peak season rate: Standard: 3000 Deluxe: 5000 Suite: 8000
Number of nights	Check if nights > 7 to apply a flat 15% long-stay discount Calculate discount = base cost x 0.15 Calculate final price = base cost - discount	Final total Price for the guest	Discount Rule: If night > 7, discount = 15%

2. IPO Chart

Input	Process	Output
- Season - Room type - Nights	1. Read input values 2. Match season and room type using nested decision structures to set rate 3. Compute: $\text{base-cost} = \text{rate} * \text{nights}$ - If night > 7, set discount = base-cost * 0.15 - - Else: set discount = 0 4. Compute: $\text{final price} = \text{base cost} - \text{discount}$ 6. Output the calculated values	- Rate - Discount - final price

Algorithm:

1. Start
2. Input season (1 for Peak, 2 for Off-Peak)
3. Input room-type (1 for Standard, 2 for Deluxe, 3 for suite)
4. Input number of nights -
5. Check season:
 - If season == 1 (Peak):
 - If room type == 1: set rate = 5000
 - Else if room-type == 2: set rate = 8000
 - Else if room-type == 3: rate = 12000
 - Else room type $\neq$ 3 "invalid room"
 - Else if season == 2 (Off Peak):
 - If room-type == 1: rate 3000
 - Else if room type == 2: rate 5000
 - Else if room type == 3, rate 8000
 - Else: Print invalid room type error
 - Else: Print invalid season selection
6. Calculate base-cost = rate * nights.
7. If nights > 7:
 - Calculate discount = base-cost * 0.15
 - Else
 - Set discount = 0.0
8. Calculate final Price = base cost - discount
9. Display rate, discount and final price
10. End -

4. Pseudocode:

Begin

Declare integer season, room type, nights
Declare Real rate, base-cost, discount,
final-price

Print "Enter season (1 for Peak, 2 for Off-Peak):"

Read season

Print "Enter Room type (1 for standard, 2 for Deluxe, 3 for suite)."

Read Room type

Print "Enter Number of nights:"

Read nights.

If season == 1 then

 If room-type == 1 then

 rate = 5000

 Else if room type == 2 then

 rate = 8000

 Else if room-type == 3 then

 rate = 12000

 else

 Print "Invalid Room type selection"

 Exit.

 end if

Else if season == 2 then

 if room-type == 1 then

 rate = 3000

 else if room-type == 2 then

 rate = 8000

 else if room-type == 3 then

 rate = 8000

 else

 Print "Invalid Room type selection"

 end if

 Exit

Else

Print "Invalid season selection"-

Exit

End if

base-cost = rate * nights

If nights > 7 Then

discount = base-cost * 0.15

Else

discount = 0.0

End if

final-price = base-cost - discount

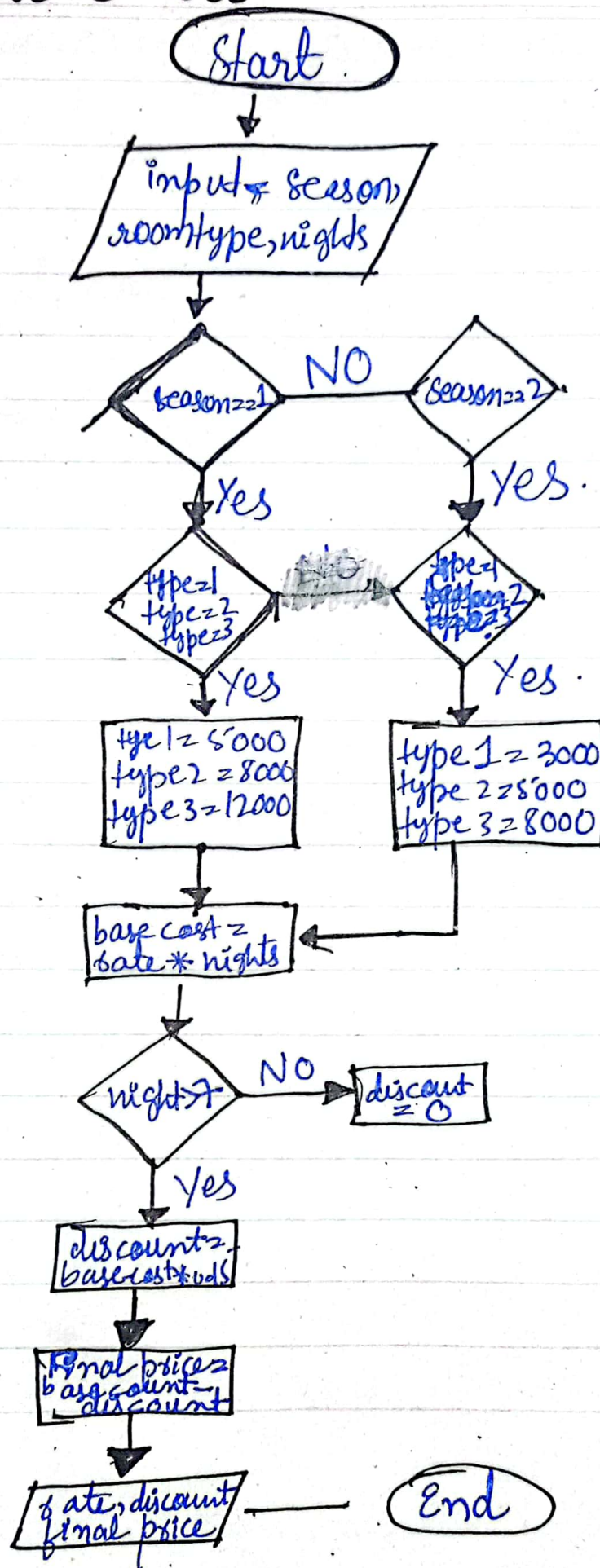
Print "Nightly Rate: Rs. ", rate

Print "Discount Applied: Rs. ", discount

Print "Final Price: R. ", final-price

End

Flow chart.



Question 02:

PAC:

Data input	Processing	Output	Given
- Current floor (current floor) - Requested floor	1. Compare targeted floor with current floor - 2. If targeted floor > current floor, status is "moving up" - 3. else if targeted floor < current floor, status is move down - 4. else targeted floor = current floor, status is Doors open - 5. Update: Current floor to target - floor	• Movement status ("Moving up", "Moving down" or "Doors Open". • Updated Current floor	• Initial floor starts at 0 (Ground floor) • single request processing per decision step.

IPO:

Input	Process	Output
• Current - floor • target - floor	1. Read Current floor and target - floor 2. Evaluate decision condition using if-else structure. 3. Set direction message. 4. assign current - floor target - floor 5. Print status and updated floor -	• message • Current - floor

Algorithm:

1. Start
2. Input current-floor and targeted-floor.
3. If targeted-floor $>$ current-floor:
 - set action = "Moving up".
4. Else if target-floor $<$ current-floor:
 - set action = "Moving down".
5. Else
 - set action = "Doors opening".
6. Update current-floor = target-floor
7. Display action and new current-floor.
8. End.

Pseudocode:

Begin

Declare integer current-floor, target-floor.

Declare string action.

Print "Enter Current floor: "

Read current-floor

Print "Enter Requested floor: "

Read target-floor

If target-floor $>$ current-floor then
 action = "Moving up"

Else if target-floor $<$ current-floor then
 action = "Moving down".

Else
 action = "Doors opening".

Endif

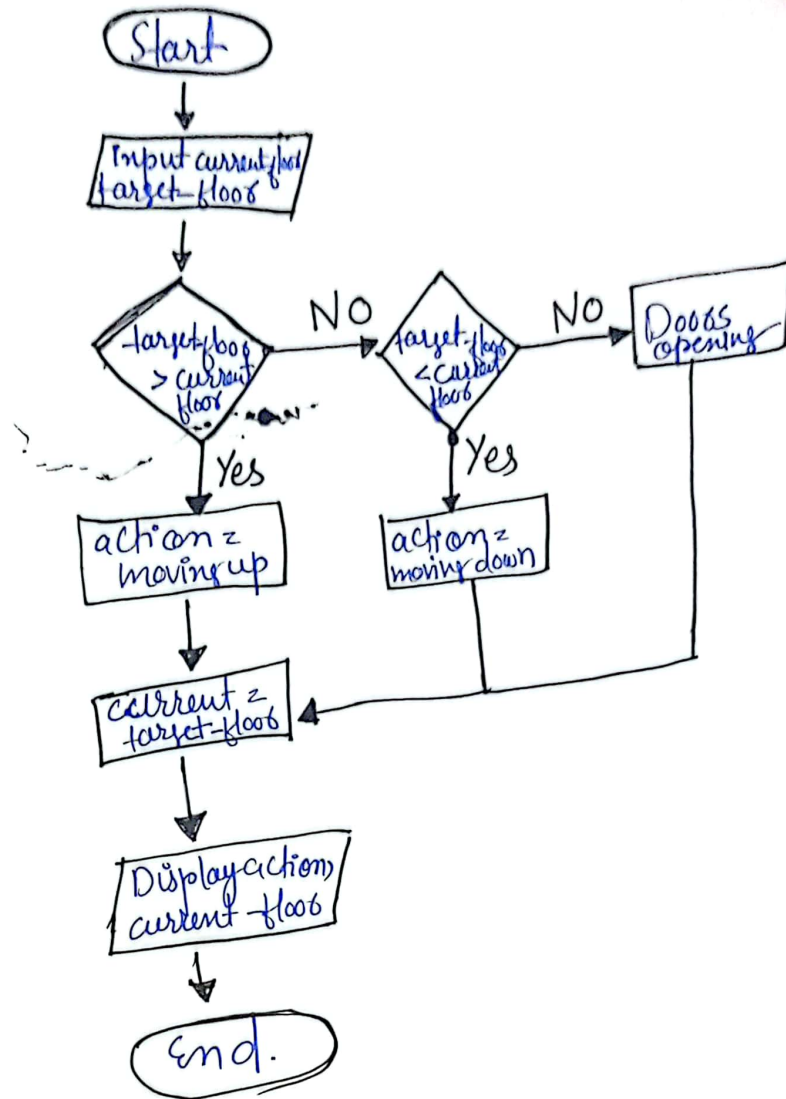
current-floor = target-floor

Print "Status: ", action

Print "Updated current floor:", current-floor

End -

Flow chart .



Qno:3. (PAC)

Input	Processing	Output	Given
Marks for 5 subjects $(m_1, m_2, m_3, m_4, m_5)$	1. Calculate $total = m_1 + m_2 + m_3 + m_4 + m_5$ 2. Calculate $average = total / 5$ 3. Check if any single marks is ≥ 33 4. Determine result: If any mark < 33 : "fail - Subject deficiency" Else if $average \geq 80$: Distinction Else if $average \geq 60$: Pass Else fail -	Total marks Average marks Final Result / Classification	Minimum 33 passing marks per subject. Classification Criteria: • $average \geq 80$: Distinction • $60 \leq average < 80$: Pass • $average < 60$: Fail Subject failure override applies if any single sub < 33

IPO:

Input	Process	Output
m_1, m_2, m_3, m_4, m_5	- Read marks of 5 subjects - Calculate total and average. - Check condition for failed individual subjects ($m_1 < 33$) - Average ≥ 60 "Pass"	- Total Marks - Average marks - Final Result/Classification-

Algorithm:

- Start.
- Input marks for 5 subjects (m_1, m_2, m_3, m_4, m_5)
- Compute $\text{total} = m_1 + m_2 + m_3 + m_4 + m_5$
- Compute $\text{average} = \text{total} / 5$
- If ($m_1 < 33$ OR $m_2 < 33$ OR $m_3 < 33$ OR $m_4 < 33$ OR $m_5 < 33$)
 - Set result = "Fail - Subject deficiency"
- Else if $\text{average} \geq 80$:
 - Set result = "Distinction"
- Else if $\text{average} \geq 60$:
 - Set result = "Pass"
- Else
 - Set result = "Fail"
- Display total, average, and result.
- End.

Pseudocode:

Begin

Declare Real m_1, m_2, m_3, m_4, m_5 ; total average

Declare String result -

Print "Enter marks for 5 subjects (out of 100):"

Read m_1, m_2, m_3, m_4, m_5 -

$total = m_1 + m_2 + m_3 + m_4 + m_5$

$average = total / 5.0$

If $m_1 < 33$ or $m_2 < 33$, $m_3 < 33$, $m_4 < 33$ or $m_5 < 33$ Then result = "Fail - Subject Deficiency"

Else If average ≥ 80 then
result = "Distinction"

else if average ≥ 60 then
result = "Pass"

else
result = "Fail"

End if

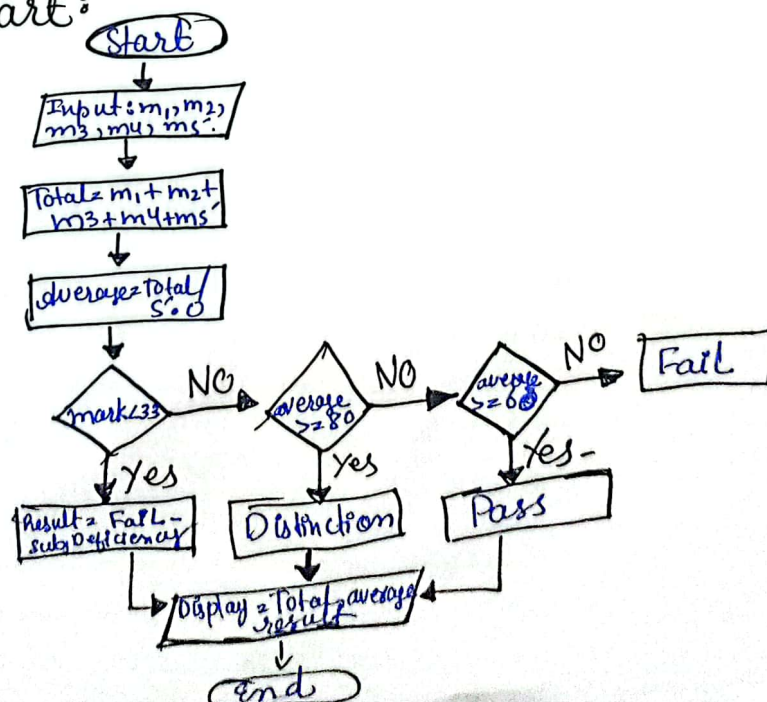
Print "Total Marks: ", total

Print "Average: ", average

Print "Final Result: ", result

End -

Flow Chart:



Qno: 4

PAC -

Input	Process	Output	Given
- Quantity (q) - Price per item (p) - Discount Percentage (d) - Tax Percentage (t)	1. Validate inputs $q > 0, p > 0,$ $d \geq 0, t \geq 0$ 2. Calculate subtotal $s = q \times p$ 3. Calculate Discounted Amount $a = s - \frac{s \times d}{100}$ 4. Calculate Final Bill $f = a + \frac{a \times t}{100}$	- Subtotal (s) - Discounted Amount (a) - Final Bill (f)	$s = q \times p$ $a = s - \frac{s \times d}{100}$ $f = a + \frac{a \times t}{100}$ Validation: Inputs must be non-negative (quantity and price strictly positive) -

IPO:

Input	Process	Output
q, p, d, t	1. Prompt and read inputs. 2. Check for invalid values using if conditions. 3. Compute s, a and f sequentially. 4. Output generated bill document.	- Subtotal - Discounted Amount - Tax amount - Final Bill Amount

Algorithm.

1. Start.
2. Input quantity q , price per item p ; discount percentage d , tax percentage t .
3. If $q < 0$ OR $p < 0$ OR $d < 0$ OR $t < 0$ -
Display error message "Invalid input values" and exit.
4. Calculate subtotal $s = q \times p$.
5. Calculate discounted amount $a = s - \frac{s \times d}{100}$.
6. Calculate final bill $f = a + \frac{a \times t}{100}$.
7. Display generated bill discount (s , a tax amount, and f).
8. End.

Pseudocode

Begin

Declare Real q, p, d, t, s, a , final-bill.

Print "Enter quantity of products: "

Read q

Print "Enter price per item: "

Read p

Print "Enter discount percentage: "

Read d

Print "Enter tax percentage: "

Read t

If $q < 0$ OR $p < 0$ OR $d < 0$ OR $t < 0$ Then
Print "Error: Invalid input entered,"

Exit.

End If.

$s = q \times p$, $a = s - (s \times d / 100)$, Final-Bill $= a + (a \times t / 100)$

Print "Shopping Bill"

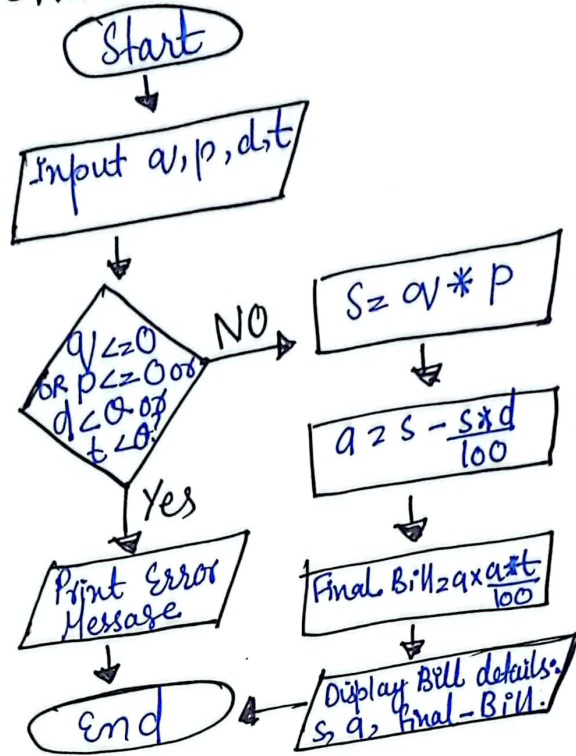
Print "Subtotal: ", s

Print "Discounted amount: ", a

Print "Final Bill amount: ", Final-Bill

End-

Flow Chart:



Qno: 5-

Algorithm:

1. Start
2. Input vehicle ('C' for Car, 'B' for Bike, 'V' for Van)
3. Input category ('F' for faculty, 'S' for student, 'G' for Guest)
4. Input permit ('Y' for Yes, 'N' for No)
5. If permit != 'Y':
 - Display "Access Denied: Valid permit required".
6. Else
 - If category == 'F' (faculty)
 - Assigned to Zone A.
 - Else if category == 'S' (student):
 - if vehicle == 'V' (Van)
 - Assigned / Redirected to Zone C.
 - Else:
 - Assigned to Zone B.
 - Else if category == 'G' (Visitor/Guest)
 - Assigned to Zone C.
 - Else:
 - Display "Invalid User Category".
7. End.

PAC:

Data /	Processing	Output	Given-
- Vehicle Type: ('C', 'B', 'V') - User Category: ('F', 'S', 'G') - Valid permit: ('Y', 'N')	1. Validate inputs vehicle in ('C', 'B', 'V') category in ('F', 'S', 'G') Permit in ('Y', 'N') 2. Check access condition using nested if-else statements: 3. Faculty F + Permit Y: assigned Zone A (if Van, Zone A if space available) • Student (S) + Permit (X): assignment Zone B (if Van, Zone B if space available) redirected to Zone C. • Guest/visitor ('G') + Permit (Y): assigned Zone C. • Permit 'W': Access Denied.	• assigned Parking Zone (zone A, B, C) OR access denied / Error Message.	Zone Capacities: • Zone A (Faculty): Max 20 • Zone B (Students): Max 40 • Zone C (visitors): Max 15 Special Rules: • Student in Van → Redirect to Zone C • Valid permit mandatory -

IPC:

Input	Process	Output
- Vehicle - Category - permit-	1. Read input parameters 2. Evaluate entry permit status - 3. Determine parking zone using nested decision-making logic. 4. Display assigned or access rejection message.	assigned Zone OR Rejected Message-

4. Pseudocode.

Character vehicle, category, permit -

Print "Enter Vehicle type (C: Car, B: Bike, V: Van): "

Read Vehicle

Print "Enter User Category (f: faculty, s: student, G: Guest)

Read category

Print "Valid Permit? (Y/N): "

Read permit

If permit != 'Y' AND permit != "y" then

Print "Access Denied: Valid permit is required."

Else

If category == 'f' OR category == 'F' Then

Print "Access Granted: assigned to Zone A (Faculty Zone) -

else if category == 's' OR category == 'S' Then

If vehicle == 'V' or vehicle == 'v' then

Print "Access Granted: Student Van redirected to zone C -

Else

Print "Access Granted: assigned to Zone B -

End if

Else if category == 'G' OR category == 'g' Then

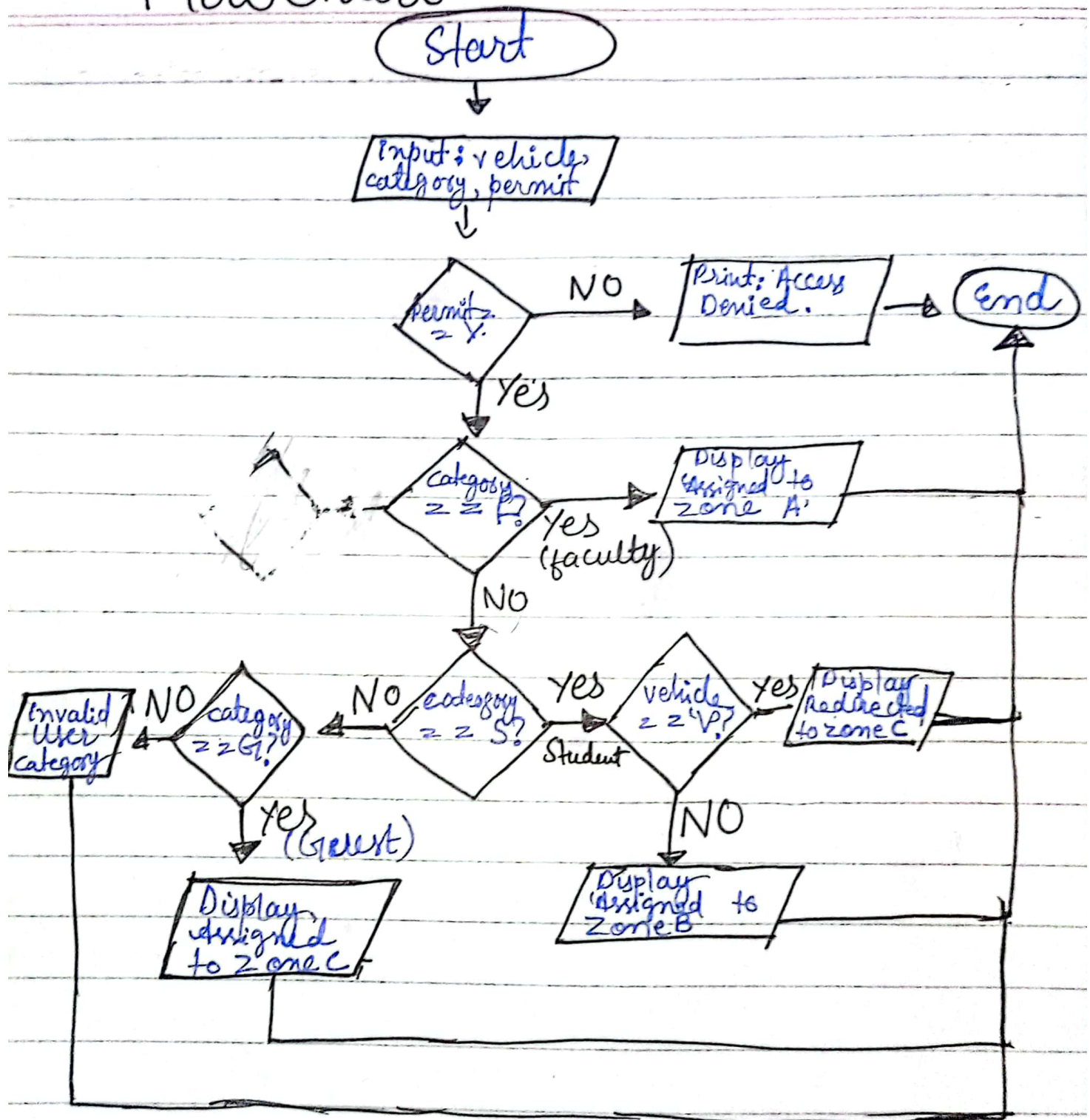
Print "Access Granted: assigned to zone C (visitor zone)

Else

Print "Invalid User Category Entered."

End.

Flowchart



Qno: 6

PAC

Input	Process	Output	Given
Variable type (E or H)	Availability check: if unavailable (charging unavailable - Parking	Off Peak Vehicle type or Battery status - Required charging %	Off Peak (< 17 or > 22) Rs. 35 (Discount 20%)
Battery SOC (soc: %)	only for H, No charging slot for E. E qualifies, H qualifies only if soc < 40 .		Peaks ($17 \leq \text{time}$ ≤ 22) Rs. 50 Discount = 10%
Target level (target: %)	Req - charge = target - soc, if target $\leq$ soc, NO charging required	Priority level or Peak/ Off Peak status	Emergency Priority: No memberships discount on charging
Parking duration: hrs	Priority 1: soc ≤ 15 AND target $\geq 80 \rightarrow$ Emergency charging	Charging cost	
Current time: 24-hrs, integer	Priority 2: Disable = 'Y' OR member = 'X' AND soc ≤ 30 "Priority"	Parking cost or discounts	Disabled priority 100% free parking (does not apply to charging)
Parking membership: Y/N	Priority 3: Otherwise Normal charging	Final Payable Amount	Member: 20% parking discount
Disabled priority disable (Y/N)		Duration Warning Message	

(IPO)

	Input	Process	Output
9.	• Vehicle,	1. Prompt and read driver inputs.	• Vehicle-type
10	• Soc, target,	2. Perform decision checks for station availability and vehicle eligibility.	• Soc, target,
	duration,	3. Determine priority tier via multi-condition statements.	req-charge
	time, member,	4. Determine time slot (Peak vs Off Peaks) and compute base charging cost.	• priority-status
	disabled,	5. Parking cost based on duration and apply parking discounts.	• time-status
11	available	6. Sum total payable amount.	• charging-cost
12			• Parking-cost
			• final-payable
			• warning-msg

Algorithm:

1. Start.
2. Input Vehicle (E or H) soc, target, duration, time member (Y/N), disabled (Y/N) available (Y/N)
3. If available == 'N':
 - if vehicle == 'H', print "Charging unavailable parking only" -
 - else, print "No charging slot available" -
4. If vehicle == H AND soc > 40:
 - Print "Vehicle does not qualify for EV charging"
5. Calculate req-charge = target - soc -
6. If req-charge <= 0:
 - Print "No charging required" -
 - set charging-cost = 0.0

7. Else

- Determine priority:

- If $\text{soc} \leq 15$ AND $\text{target} \geq 80$: set priority = 1 ("Emergency charging priority").

- Else if $\text{disabled} = 'Y'$ OR ($\text{member} = 'Y'$ AND $\text{soc} \leq 30$): set priority = 2 "Priority charging".

- Else: set priority = 3 ("Normal charging").

- Determine Charging Rate or Discount:

- if $\text{time} \geq 17$ AND $\text{time} \leq 22$ (Peak Hours).

- set base-rate = 50.0

- If $\text{member} = 'Y'$ AND $\text{priority} \neq 1$:
set c-discount = 0.1

- Else: set c-discount = 0.0.

- Else (Off-Peak Hours):

- set base-rate = 35.0.

- If $\text{member} = 'Y'$ AND $\text{priority} \neq 1$:
set c-discount = 0.2.

- Else: set c-discount = 0.0

- Compute:

$$\text{charging-cost} = \text{rev-charge} * \text{base-rate} * (1.0 - \text{c-discount}).$$

8 Calculate Parking cost:

- If $\text{duration} \leq 2$: set base-park = 200.0.

- Else if $\text{duration} \leq 5$: set base-park = 400.0

- Else: set base-park = 700.0

Apply parking discounts.

- If $\text{disabled} = 'Y'$: set parking cost = 0.0

- Else if member == 'X': Set parking-cost = base-park * 0.80.
- Else: Set parking-cost = base-park-
- 9. Calculate final-payable = charging-cost + parking-cost -
- 10. Determine Warning:
 - If duration > 8: Set warning = "Long-stay warning: Please relocate your vehicle after charging"
 - Else: Set warning = "Standard parking duration"
- 11. Display summary report
- 12. End -

Pseudocode:

Begin

Read vehicle, soc, target, duration, time, members availability & qualification

if available == 'N' then

if vehicle == H then print "Charging unavailable parking only"

Else print "No charging slot available"

End if

if vehicle == H and soc < 40 then

Print "Vehicle does not qualify for EV charging"

End if

! Charging calculation

req-charge = target-soc

```

If req-charge  $\leq 0$  then
    charging-cost = 0.0
Else if available == X then
    ! Determine priority
    if soc  $\leq 15$  AND target  $\geq 80$  then
        priority = 1 ! Emergency (No-discount)
    Else if disabled == Y or (member == 'X' AND
        soc  $\leq 30$ ) then
        priority = 2 ! Priority.
    Else
        priority = 3 ! Normal
    End if
    ! Set Rates & member discounts
    if time  $\geq 17$  and time  $\leq 22$  then
        rate = 50.0
        c-disc = (member == X AND priority  $\neq 1$ )
    Else
        rate = 35.0
        c-disc = (member == X AND priority  $\neq 1$ )
    End if
    Charging-cost = req-charge * rate * (1 - c-disc)
End if.
! Parking fee logic
if duration  $\leq 2$  then base-park = 200.0
Else if duration  $\leq 5$  then base-park = 400.0
Else base-park = 700.0
End if
Print vehicle, soc, req-charge, charging-cost,
parking-cost, final-payable, warning
End-

```

Flow Chart

